**Date:** 6 Mar 2026

**Project Name:** UCab - Modern Ride-Sharing Platform

**Maximum Marks:** 4 Marks

**Project Design Phase-II**

**Solution Requirements (Functional & Non-functional)**

**Functional Requirements**

Following are the functional requirements of the proposed solution (based on actual implementation).

| FR No. | Functional Requirement (Epic) | Sub Requirement (Story / Sub-Task) |
|---|---|---|
| **FR-1** | User Authentication & Registration | • Registration through Custom Form (Name, Email, Password, Role)<br>• Login via Email and Password<br>• Secure Session Management via JWT |
| **FR-2** | Ride Booking System | • Manual Location Input (Pickup and Drop-off text fields)<br>• Cab Type Selection (Mini, Sedan, SUV) with predefined pricing<br>• Simulated Ride Search Delay (UX enhancement) |
| **FR-3** | Payment Interface | • Display Static QR Code (qrcode.png) for UPI Payments |

| | | |
|---|---|---|
| | | • Manual "Payment Completed" Confirmation Button<br><br>• Pre-calculated Fare Display based on selected Cab Type |
| **FR-4** | Ride Tracking & Interface | • Display Assigned Driver Details (Name, Rating, Vehicle Info)<br><br>• Simulated Live Tracking UI with Map Placeholder<br><br>• WebSocket (Socket.io) infrastructure setup for future live updates |
| **FR-5** | User Dashboard & History | • View Recent Activity and Ride History (UI)<br><br>• Track Total Expenditure (in Rupees)<br><br>• View Ride Status Badges (Completed / Cancelled) |

**Non-functional Requirements**

Following are the non-functional requirements of the proposed solution.

| NFR No. | Non-Functional Requirement | Description |
|---|---|---|
| **NFR-1** | Usability | The application features a highly intuitive, responsive, and mobile-first UI using **Bootstrap** and custom CSS, ensuring a seamless experience across all devices. |
| **NFR-2** | Security | User passwords are securely hashed using **Bcrypt.js** before storing in the MongoDB database. API routes are protected using **JWT (JSON Web Tokens)**. |
| **NFR-3** | Reliability | The Express.js backend provides reliable REST API endpoints for user authentication and data retrieval, ensuring consistent communication between the React frontend and MongoDB. |
| **NFR-4** | Performance | The frontend is built with **Vite**, ensuring extremely fast hot-module replacement during development and optimized, lightweight static assets for production. |
| **NFR-5** | Availability | The MERN stack architecture allows the backend and frontend to be deployed on cloud platforms (like Render or Railway) to ensure high availability for users. |
| **NFR-6** | Scalability | The authentication system is stateless (via JWT), allowing the Node.js server to scale horizontally without session memory issues. |